
Modül 13 — Reactive Programlama ve WebFlux

Ders Notları

Spring Boot Edu



2026-09-24

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Bloklayan ve Bloklamayan	2
2.2 Mono, Flux ve Abonelik	3
2.3 Backpressure	3
3. Adım Adım Örnekler	3
3.1 Operatörler	3
3.2 Hatalar ve Yeniden Denemeler	4
3.3 Testlerde Zaman ve Backpressure	5
3.4 Functional Endpoint'ler ve R2DBC	5
3.5 Reactive MongoDB ve Mono.zip	7
3.6 Server-Sent Events	7
3.7 Reactive mi, Virtual Thread mi?	8
3.8 Test	10
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	11
6. İleri Okuma	11

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- “Reactive”in ne demek olduğunu açıklamak: backpressure’lı, bloklamayan, asenkron akışlar
- Mono ve Flux ile çalışmak: oluşturmak, dönüştürmek, birleştirmek, hataları yönetmek ve yeniden denemek
- Reactive kodu, sanal zaman (virtual time) dahil, StepVerifier ile test etmek
- Functional endpoint’ler ve annotation’lı controller’larla WebFlux API’leri kurmak
- PostgreSQL’e R2DBC ile, MongoDB’ye reactive olarak erişmek
- Server-Sent Events ile tarayıcılara canlı veri göndermek
- Reactive kod ile virtual thread’ler arasında karar vermek

Ön koşullar: Modül 03 (REST) ve 07 (MongoDB) · **Tahmini süre:** 6 saat · **Docker gerekir**

2. Kavramlar

2.1 Bloklayan ve Bloklamayan

Klasik bir Spring MVC uygulaması istek başına bir thread kullanır. Kod veritabanını beklerken thread de bekler. **Bloklamayan** (non-blocking) bir uygulama ise “yanıt gelince beni çağır” diye kaydolur ve thread’i serbest bırakır. Böylece birkaç event-loop thread’i binlerce eşzamanlı isteğe hizmet edebilir.

	Spring MVC (bloklayan)	Spring WebFlux (reactive)
Sunucu	Tomcat, istek başına bir thread	Netty, birkaç event-loop thread’i
Dönüş tipleri	Book, List<Book>	Mono<Book>, Flux<Book>
Veritabanı erişimi	JDBC, JPA	R2DBC, reactive MongoDB sürücüsü
Kod stili	adım adım	operatörlerden oluşan bir boru hattı

! Önemli

Bir event-loop thread’inde **asla bloklamamalısınız**: Thread.sleep yok, JDBC yok, block() yok. Tek bir bloklayan çağrı o thread’deki tüm istekleri durdurur.

2.2 Mono, Flux ve Abonelik

- Bir `Mono<T>` **0 veya 1** eleman yayar, sonra tamamlanır (veya hata verir).
- Bir `Flux<T>` **0 ile n** arası eleman yayar, sonra tamamlanır (veya hata verir).
- Bir `Mono` veya `Flux` oluşturmak hiçbir şey yapmaz. İş ancak biri **abone olduğunda** (subscribe) başlar. `WebFlux`'ta controller'ınızın döndürdüğüne framework abone olur.

2.3 Backpressure

Abone, yayıncıya kaç eleman işleyebileceğini söyler (`request(n)`). Böylece yavaş bir tüketici, hızlı bir üretici tarafından boğulmaz. Bu sözleşme, `Reactor`'ın uyguladığı `Reactive Streams` spesifikasyonundan gelir.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. PostgreSQL ve MongoDB kök dizindeki `compose.yaml` dosyasından başlar:

```
./mvnw -pl modules/13-reactive/lesson spring-boot:run
```

Veri ayarları (spring: altında):

```
# All modules share the "bookstore" database of compose.yaml; this one uses the schema
↪ "reactive"
# (R2DBC side: see book/R2dbcSchemaConfiguration).
flyway:
  schemas: reactive                                # Flyway (JDBC) creates the tables before the app
  ↪ starts
mongodb:
  database: bookstore_reactive
```

3.1 Operatörler

```
public Flux<String> titlesInCapitals() {
    return Flux.just("Effective Java", "Spring in Action", "Java Puzzlers")
        .map(String::toUpperCase);                // runs only when
        ↪ someone subscribes
}

public Flux<Double> pricesOf(String... isbn) {
```

```

return Flux.fromArray(isbns)
    .flatMap(this::lookupPrice);           // all lookups run at
    ↪ the same time
}

Mono<Double> lookupPrice(String isbn) {
    Double price = PRICES.get(isbn);
    return price == null
        ? Mono.error(new NoSuchElementException("No price for " + isbn))
        : Mono.just(price).delayElement(Duration.ofMillis(100)); // a slow remote
    ↪ call, without blocking
}

```

- map her elemanı senkron olarak dönüştürür.
- flatMap her elemanı yeni bir yayıncıya çevirir ve onları **eşzamanlı** çalıştırır. Sonuçlar girdinin sırasıyla değil, bitme sırasıyla gelir. Tur prices (in parallel): [95.0, 89.9] yazdırır, oysa önce 9780134685991 (89.90) istenmişti. Sıra önemliyse concatMap veya flatMapSequential kullanın.
- delayElement bir thread'i bloklamadan bekler.

StepVerifier abone olur ve her sinyali kontrol eder:

```

StepVerifier.create(basics.titlesInCapitals())
    .expectNext("EFFECTIVE JAVA", "SPRING IN ACTION", "JAVA PUZZLERS")
    .verifyComplete();

```

3.2 Hatalar ve Yeniden Denemeler

Bir hata da bir eleman gibi bir sinyaldir. Boru hattında aşağı ilerler ve bir operatör onu ele almazsa hattı sonlandırır:

```

public Mono<Double> priceOrFallback(String isbn) {
    return lookupPrice(isbn)
        .onErrorResume(NoSuchElementException.class, e -> Mono.just(0.0)); //
    ↪ errors are signals too
}

public Mono<Double> flakyPriceWithRetry() {
    var attempts = new AtomicInteger();
    return Mono.defer(() -> attempts.incrementAndGet() < 3
        ? Mono.<Double>error(new IllegalStateException("timeout"))
        : Mono.just(89.90))
        .retryWhen(Retry.backoff(3, Duration.ofMillis(10))); // resubscribe: 10
    ↪ ms, 20 ms, ...
}

```

- `onErrorResume` hatayı başka bir yayıncıyla değiştirir, burada yedek bir fiyatla.
- `retryWhen(Retry.backoff(...))` giderek uzayan bir beklemeden sonra yeniden abone olur. `Mono.defer`, her denemenin kaynağı gerçekten yeniden çağırmasını sağlar.

3.3 Testlerde Zaman ve Backpressure

Dakikada bir eleman yayan bir akış, bir testi dakikalarca sürdürürdü. **Sanal zaman** ile `StepVerifier` simüle edilmiş bir saati iletir:

```
StepVerifier.withVirtualTime(() -> basics.tick(Duration.ofMinutes(1)).take(3))
    .expectSubscription()
    .thenAwait(Duration.ofMinutes(3))           // gerçek bekleme yok
    .expectNext(0L, 1L, 2L)
    .verifyComplete();
```

Backpressure, elemanları tek tek isteyerek test edilebilir:

```
public Flux<Integer> numbers(int count) {
    return Flux.range(1, count)
        .doOnNext(n -> produced.incrementAndGet()); // counts what was
        ↪ really produced
}
```

`ReactorBasicsTest` önce 2, sonra 1 eleman ister ve iptal eder. `Flux.range(1, 1000)` içinden tam olarak 3 eleman üretilir.

3.4 Functional Endpoint'ler ve R2DBC

R2DBC, JDBC'nin bloklamayan karşılığıdır. Spring Data bunun üzerine tanıdık repository'leri sunar:

```
public interface BookRepository extends ReactiveCrudRepository<Book, Long> {

    Mono<Book> findByIsbn(String isbn);
}
```

WebFlux, annotation'lı controller'ları (Spring MVC'deki gibi) ve yönlendirmenin sıradan kod olduğu **functional endpoint'leri** destekler:

```
@Configuration(proxyBeanMethods = false)
class BookRoutes {
```

```

@Bean
RouterFunction<ServerResponse> bookRouter(BookHandler books) {
    return route()
        .path("/api/books", builder -> builder
            .GET("", books::all)
            .GET("/{isbn}", books::one)
            .POST("", books::create))
        .build();
}

```

```

@Component
class BookHandler {

    private final BookRepository books;

    BookHandler(BookRepository books) {
        this.books = books;
    }

    Mono<ServerResponse> all(ServerRequest request) {
        return ServerResponse.ok().body(books.findAll(), Book.class); // streams
        // the rows as they come
    }

    Mono<ServerResponse> one(ServerRequest request) {
        return books.findByIsbn(request.pathVariable("isbn"))
            .flatMap(book -> ServerResponse.ok().bodyValue(book))
            .switchIfEmpty(ServerResponse.notFound().build()); // empty
        // Mono -> 404
    }

    Mono<ServerResponse> create(ServerRequest request) {
        return request.bodyToMono(Book.class)
            .flatMap(books::save)
            .flatMap(saved -> ServerResponse.created(URI.create("/api/books/" +
                saved.isbn()).bodyValue(saved)));
    }
}

```

- `ServerResponse.ok().body(flux, Book.class)` satırları veritabanından geldikçe yazar.
- `switchIfEmpty(...)` boş bir Mono'yu (böyle bir kitap yok) 404'e çevirir.

i Not

Flyway'in R2DBC desteği yoktur. Bu yüzden şemayı reactive kısım başlamadan önce

JDBC üzerinden migrate eder. Modülde PostgreSQL JDBC sürücüsünün de olmasının nedeni budur. R2DBC bağlantıları, R2dbcSchemaConfiguration içindeki bir ConnectionFactoryOptionsBuilderCustomizer ile aynı şemaya (reactive) yönlendirilir.

3.5 Reactive MongoDB ve Mono.zip

Yorumlar MongoDB’de, bir ReactiveMongoRepository içinde durur. Detay sayfası PostgreSQL’deki kitaba **ve** MongoDB’deki yorumlarına ihtiyaç duyar. Mono.zip ikisine aynı anda abone olur ve sonuçları birleştirir:

```
@GetMapping("/details")
Mono<ResponseEntity<BookDetails>> details(@PathVariable String isbn) {
    return Mono.zip(books.findByIsbn(isbn), // PostgreSQL ...
                   reviews.findByIsbnOrderByStarsDesc(isbn).collectList() // ... and
    ↪ MongoDB, at the same time
                   .map(both -> ResponseEntity.ok(toDetails(both.getT1(), both.getT2())))
                   .defaultIfEmpty(ResponseEntity.notFound().build()); // no book → zip
    ↪ is empty → 404
}
```

- İki sorgu paralel çalışır: Yanıt süresi toplam değil, yavaş olanın süresidir.
- Mono’lardan biri boşsa (böyle bir kitap yok) zip de boştur ve defaultIfEmpty bunu 404’e çevirir.

```
curl localhost:8080/api/books/9780134685991/details
{"isbn":"9780134685991","title":"Effective
↪ Java","price":89.90,"averageStars":0.0,"reviews":[]}
```

3.6 Server-Sent Events

Server-Sent Events (SSE) ile sunucu HTTP yanıtını açık tutar ve bir şey olduğunda bir event yazar. Tarayıcının EventSource’u bağlantı koparsa kendisi yeniden bağlanır.

```
@RestController
@RequestMapping("/api/orders")
class OrderStreamController {

    private final Sinks.Many<OrderEvent> orders =
    ↪ Sinks.many().multicast().directBestEffort(); // hot: no replay

    @PostMapping
    @ResponseStatus(HttpStatus.ACCEPTED)
    void place(@RequestBody OrderEvent order) {
```

```

        orders.tryEmitNext(order);
    }

    @GetMapping(path = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    Flux<ServerSentEvent<OrderEvent>> stream() {
        var connected =
            ↪ ServerSentEvent.<OrderEvent>builder().comment("connected").build();    //
            ↪ sends the headers now
        return orders.asFlux()
            .map(order -> ServerSentEvent.builder(order).event("order").build())
            .startWith(connected);           // the connection stays
            ↪ open
    }
}

```

- `Sinks.many().multicast()` **sıcak** (hot) bir yayıncıdır: Her abone, abone olduğu andan itibaren event'leri görür. Geçmiş yeniden oynatılmaz.
- WebFlux yanıt header'larını ilk elemanla birlikte gönderir. `connected` yorumu, client'ın header'ları ilk siparişle değil hemen almasını sağlar. Client'lar yorumları yok sayar.

```

curl -N localhost:8080/api/orders/stream
:connected

event:order
data:{"isbn":"9780134685991","quantity":2}

```

3.7 Reactive mi, Virtual Thread mi?

Java 21, **virtual thread**'leri getirdi: Bloklamanın artık pahalı olmadığı kadar ucuz thread'ler. İki yaklaşım da aynı problemi çözer: kıt bir platform thread'ini boşa harcamadan beklemek.

```

public Result reactive(int calls){
    long start = System.nanoTime();
    Long count = Flux.range(1, calls)
        .flatMap(i -> Mono.just(i).delayElement(latency), calls)    // waits
        ↪ without a thread
        .count()
        .block();           // only here: to
        ↪ measure
    return new Result(count == null ? 0 : count.intValue(),
        ↪ Duration.ofNanos(System.nanoTime() - start));
}

```

```

public Result virtualThreads(int calls) {
    long start = System.nanoTime();
    int results = 0;
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        List<Future<Integer>> futures = new ArrayList<>();
        for (int i = 1; i <= calls; i++) {
            int call = i;
            futures.add(executor.submit(() -> {
                Thread.sleep(latency); // plain
                ↪ blocking code
                return call;
            }));
        }
        for (Future<Integer> future : futures) {
            future.get();
            results++;
        }
    } catch (InterruptedException | ExecutionException e) {
        throw new IllegalStateException(e);
    }
    return new Result(results, Duration.ofNanos(System.nanoTime() - start));
}

```

== 3.7 Reactive vs. virtual threads (200 calls of 200 ms)

reactive: 206 ms

virtual threads: 215 ms

İkisi de yaklaşık **tek** bir çağrı kadar sürer. Sırayla yapılıyadı 40 saniye sürerdi.

Virtual thread seçin, eğer ...	Reactive seçin, eğer ...
uygulama tipik bir istek/yanıt servisi ise	akışlar işliyorsanız: SSE, WebSocket, Kafka, backpressure
JDBC/JPA ve bloklayan kütüphaneler kullanıyorsanız	zincirin tamamı bloklamıyorsa (R2DBC, WebClient)
ekip basit, adım adım kod istiyorsa	birçok kaynak üzerinde zip, timeout, retry gibi operatörlere ihtiyacınız varsa

★ İpucu

Çoğu yeni Spring MVC uygulaması için `spring.threads.virtual.enabled=true`, yeni programlama modeli olmadan ölçeklenebilirliğin büyük kısmını sağlar. Reactive, verinin **aktığı** yerde parlar.

3.8 Test

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
↪ pg18").asCompatibleSubstituteFor("postgres"));

    static final MongoDBContainer MONGO = new MongoDBContainer("mongo:8.0.32");

    @Bean
    @ServiceConnection // provides R2DBC *and* JDBC (Flyway) connection
    ↪ details
    PostgreSQLContainer postgresContainer() {
        return POSTGRES;
    }

    @Bean
    @ServiceConnection
    MongoDBContainer mongoContainer() {
        return MONGO;
    }
}
```

- WebTestClient WebFlux endpoint'lerini test eder ve bir SSE akışını Flux olarak okuyabilir (OrderStreamTest).
- @DataMongoTest reactive repository'ler için de çalışır.
- Bir PostgreSQL container'ından R2DBC bağlantı bilgileri için Boot, test classpath'inde org.testcontainers:testcontainers-r2dbc ister.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Bir event-loop thread'inde çalışan reactive bir boru hattının içinde asla block(), Thread.sleep veya bir JDBC metodu çağırmayın. Sunucuyu herkes için dondurur.

- **Yapın:** Controller'lardan Mono/Flux döndürün ve abone olmayı WebFlux'a bırakın.
- **Yapmayın:** Abonelik olmadan hiçbir şeyin olmadığını unutmayın. Sonucuna kimsenin abone olmadığı bir save(...) hiçbir şey kaydetmez.
- **Yapın:** Eşzamanlılık için flatMap, sıra önemliyse concatMap kullanın.
- **Yapmayın:** Hataları sessizce yutmayın. Her onErrorResume bilinçli bir karar olmalıdır.
- **Yapın:** Gerçek beklemeler yerine StepVerifier ve sanal zamanla test edin.

- **Yapmayın:** Bir WebFlux uygulamasına bloklayan kütüphaneler karıştırmayın. Mecbursanız onları `Schedulers.boundedElastic()`'e taşıyın.

5. Özet

- Reactive kod bloklamaz: Az sayıda thread çok sayıda isteğe hizmet eder ve hiçbir şey onları bloklamamalıdır.
- Mono 0-1, Flux 0-n eleman içerir. İş yalnızca abonelikte başlar.
- Operatörler dönüştürür (map), birleştirir (flatMap, zip), hataları yönetir (onErrorResume, retryWhen) ve zamanı kontrol eder (timeout, delayElement).
- StepVerifier akışları sanal zaman ve backpressure ile adım adım test eder.
- WebFlux functional endpoint'ler ve annotation'lı controller'lar sunar. R2DBC ve reactive MongoDB veritabanı erişimini bloklamayan tutar.
- SSE, normal bir HTTP yanıtı üzerinden canlı event'ler gönderir.
- İstek/yanıt servisleri için virtual thread'ler daha basit seçimdir. Reactive, akışlara ve karmaşık asenkron birleştirmelere uyar.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Framework — Web on Reactive Stack · Functional Endpoints](#)
- [Reactor Reference Guide · Which operator do I need?](#)
- [Spring Data R2DBC](#)
- [Spring Boot — Reactive Web Applications](#)
- [MDN — Server-Sent Events](#)
- [JEP 444 — Virtual Threads](#)